Section G Tasks

Lab Task 1: Student GPA Tracker

Scenario:

A university tracking system maintains student records in a BST keyed by StudentID to retrieve GPA and semester details quickly.

Task Requirements:

Each node should contain:
- StudentID (int, unique key)
- Name (string)
- GPA (float)
- Semester (int)

Core Functionalities:

- insertStudent(int id, string name, float gpa, int sem)
- findStudent(int id)
- updateGPA(int id, float newGPA)
- displayStudentsInOrder()
- getInorderIDs()

Advanced Operations:

- findTopStudent() -> Student
- countStudentsBelowGPA(float gpa)
- getAverageGPA()

Note: Some testcases have vectors in them. You need to replace them with an appropriate list or array

Google Test Cases:

```
TEST(GPABSTTest, InsertAndFindStudent) {
        GPABST bst;
        bst.insertStudent(10, "Ali", 3.2, 4);
        bst.insertStudent(20, "Sara", 3.8, 6);
        Student s = bst.findStudent(20);
```

```
        EXPECT_EQ(s.Name, "Sara"); // Expected found student
        EXPECT_FLOAT_EQ(s.GPA, 3.8);
}

TEST(GPABSTTest, UpdateGPA) {
        GPABST bst;
        bst.insertStudent(1, "A", 2.5, 2);
        bst.updateGPA(1, 3.0);
   EXPECT_FLOAT_EQ(bst.findStudent(1).GPA, 3.0); // Expected GPA updated
}

TEST(GPABSTTest, FindTopStudent) {
        GPABST bst;
        bst.insertStudent(1, "A", 2.1, 2);
        bst.insertStudent(2, "B", 3.9, 6);
        Student top = bst.findTopStudent();
        EXPECT_EQ(top.Name, "B"); // Expected highest GPA student
}

TEST(GPABSTTest, CountStudentsBelowGPA) {
        GPABST bst;
        bst.insertStudent(1, "A", 2.0, 2);
        bst.insertStudent(2, "B", 3.5, 4);
        bst.insertStudent(3, "C", 1.8, 1);
   EXPECT_EQ(bst.countStudentsBelowGPA(3.0), 2); // Expected two students below 3.0
}

TEST(GPABSTTest, InorderSortedIDs) {
        GPABST bst;
        bst.insertStudent(3, "X", 2.0, 1);
        bst.insertStudent(1, "Y", 3.0, 2);
        bst.insertStudent(2, "Z", 3.5, 3);
        vector<int> ino = bst.getInorderIDs();
        EXPECT_EQ(ino, (vector<int>{1,2,3})); // Expected sorted IDs
}

TEST(GPABSTTest, HeightAndAverageGPA) {
        GPABST bst;
        bst.insertStudent(1, "A", 3.0, 2);
        bst.insertStudent(2, "B", 2.0, 4);
        EXPECT_EQ(bst.getHeight(), 2); // Expected height
        EXPECT_DOUBLE_EQ(bst.getAverageGPA(), 2.5); // Expected average GPA
}

TEST(GPABSTTest, DeleteStudent) {
        GPABST bst;
        bst.insertStudent(10, "D", 2.9, 3);
        bst.deleteStudent(10);
        EXPECT_EQ(bst.findStudent(10).Name, ""); // Expected deleted
```

}

Lab Task 2: Binary Search Tree Compression and Reconstruction

You are part of a development team working on a **distributed storage system**.
In this system, data structures like Binary Search Trees (BSTs) need to be **transmitted across a network** (from one machine to another) or **stored on disk** for later retrieval.

Transmitting the entire pointer-based BST directly is inefficient and unreliable because:

- It contains memory addresses that have no meaning on another machine.
- It consumes large amounts of space.
- It is hard to reconstruct exactly as it was.

To make this possible, you must design an **encoding (compression) and decoding (reconstruction)** scheme for the BST — one that can:

- Convert a BST into a compact, serialized sequence (e.g., a string or list).
- Reconstruct the exact same BST structure from that sequence.

This is sometimes called **BST serialization** and **deserialization**, but here we emphasize the **"compression"** aspect — optimizing how the tree is represented.

Data Model

Each BST node stores:

Key (int) → The BST ordering key
Value (string) → Associated data (e.g., filename, user info, etc.)
Left, Right Pointers → Child nodes

BST Compression (Serialization)

You need to convert the BST into a **compact linear form** (array) that can be transmitted or stored.

Requirements:

- Use **Preorder Traversal (Root → Left → Right)** for serialization.
- Output only the `key` values (and optionally value fields).
- Do **not** include explicit `NULL` markers unless necessary.
- Optimize for space — the goal is "compression," not just any serialization.

BST Reconstruction (Deserialization)

Given the compressed sequence, reconstruct the **identical BST**.

Rules:

- The order of insertion must produce the original BST structure.
- The reconstruction must work for both balanced and skewed trees.
- Time complexity target: **O(n)** (use preorder + BST constraints, not naive re-insertion $O(n^2)$).

Verification Function

To ensure correctness, provide a function that verifies that:

- The reconstructed BST has the same in-order traversal as the original.
- Both trees have the same structure and values.

You are also expected to create the basic insertion, deletion, traversal functions that are integral to BST implementations

Test Cases

```
#include "gtest/gtest.h"

#include "BST.h"

#include <string>

#include <sstream>
```

```cpp
#include <algorithm>




// Utility: Compare two BSTs for structural + key equality
bool areIdentical(BSTNode* a, BSTNode* b) {
    if (!a && !b) return true;
    if (!a || !b) return false;
    return (a->key == b->key) &&
           areIdentical(a->left, b->left) &&
           areIdentical(a->right, b->right);
}


TEST(BSTCompressionTest, BasicInsertionAndTraversal) {
    BST tree;
    tree.insert(50, "A");
    tree.insert(30, "B");
    tree.insert(70, "C");
    tree.insert(20, "D");
    tree.insert(40, "E");
    tree.insert(60, "F");
    tree.insert(80, "G");
```

```
    std::ostringstream out;
    inorderTraversal(tree.root, out);
    std::string expected = "20 30 40 50 60 70 80 ";
    EXPECT_EQ(out.str(), expected);
}
```

```
TEST(BSTCompressionTest, CompressionProducesCorrectPreorder) {
    BST tree;
    int keys[] = {50, 30, 20, 40, 70, 60, 80};
    const int n = sizeof(keys)/sizeof(keys[0]);
    for (int i = 0; i < n; i++)
        tree.insert(keys[i], "data");

    std::string compressed = tree.compressBST();
    // Expected preorder: 50 30 20 40 70 60 80
    EXPECT_EQ(compressed, "50 30 20 40 70 60 80 ");
}

TEST(BSTCompressionTest, ReconstructionFromPreorder) {
    BST original;
    int keys[] = {50, 30, 20, 40, 70, 60, 80};
    const int n = sizeof(keys)/sizeof(keys[0]);
```

```
    for (int i = 0; i < n; i++)
        original.insert(keys[i], "x");

    // Compress → Reconstruct
    std::string serialized = original.compressBST();
    BST reconstructed = BST::reconstructBST(serialized);

    EXPECT_TRUE(areIdentical(original.root, reconstructed.root));

    // Also verify in-order traversal
    std::ostringstream a, b;
    inorderTraversal(original.root, a);
    inorderTraversal(reconstructed.root, b);
    EXPECT_EQ(a.str(), b.str());
}
```

```
TEST(BSTCompressionTest, ReconstructionSkewedRight) {
    BST t1;
    int keys[] = {10, 20, 30, 40, 50};
    const int n = sizeof(keys)/sizeof(keys[0]);
    for (int i = 0; i < n; i++)
        t1.insert(keys[i], "y");
```

```
    std::string seq = t1.compressBST();
    BST t2 = BST::reconstructBST(seq);

    EXPECT_TRUE(areIdentical(t1.root, t2.root));

    std::ostringstream out;
    inorderTraversal(t2.root, out);
    EXPECT_EQ(out.str(), "10 20 30 40 50 ");
}

TEST(BSTCompressionTest, ReconstructionSkewedLeft) {
    BST t1;
    int keys[] = {50, 40, 30, 20, 10};
    const int n = sizeof(keys)/sizeof(keys[0]);
    for (int i = 0; i < n; i++)
        t1.insert(keys[i], "z");

    std::string seq = t1.compressBST();
    BST t2 = BST::reconstructBST(seq);

    EXPECT_TRUE(areIdentical(t1.root, t2.root));
```

```
    std::ostringstream out;

    inorderTraversal(t2.root, out);

    EXPECT_EQ(out.str(), "10 20 30 40 50 ");

}
```

```
TEST(BSTCompressionTest, EmptyTreeSerialization) {

    BST tree;

    std::string result = tree.compressBST();

    EXPECT_EQ(result, "");

}
```

```
TEST(BSTCompressionTest, SingleNodeReconstruction) {

    BST t1;

    t1.insert(100, "root");



    std::string seq = t1.compressBST();

    BST t2 = BST::reconstructBST(seq);



    EXPECT_TRUE(areIdentical(t1.root, t2.root));



    std::ostringstream out;
```

```
    inorderTraversal(t2.root, out);

    EXPECT_EQ(out.str(), "100 ");

}
```

```
TEST(BSTCompressionTest, RandomTreeIntegrity) {

    BST t1;

    int keys[] = {40, 10, 30, 60, 50, 70, 20};

    const int n = sizeof(keys)/sizeof(keys[0]);

    for (int i = 0; i < n; i++)

        t1.insert(keys[i], "data");



    std::string seq = t1.compressBST();

    BST t2 = BST::reconstructBST(seq);



    EXPECT_TRUE(areIdentical(t1.root, t2.root));



    std::ostringstream a, b;

    inorderTraversal(t1.root, a);

    inorderTraversal(t2.root, b);

    EXPECT_EQ(a.str(), b.str());

}
```

```
TEST(BSTCompressionTest, CorruptedInputHandling) {

    std::string corrupted = "50 30 X 20";  // invalid token

    BST reconstructed = BST::reconstructBST(corrupted);



    // Depending on student implementation, tree may be empty or partial.

    EXPECT_TRUE(reconstructed.root == nullptr || reconstructed.root->key
== 50);

}
```